

Heitor Gabriel Barroso Reis
163793

Design e Implementação de um Processador de arquitetura ARM com Utilização do kit FPGA

São José dos Campos - SP, Brasil
Setembro de 2024

Heitor Gabriel Barroso Reis
163793

Design e Implementação de um Processador de arquitetura ARM com Utilização do kit FPGA

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores.

Docente: Prof. Dr. Sérgio Ronaldo Barros dos Santos

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - SP, Brasil

Setembro de 2024

Resumo

Na disciplina de Arquitetura e Organização de Computadores, foram abordados vários conceitos sobre o projeto e design de processadores, desde periféricos até componentes internos como memória e processadores. O objetivo deste projeto é desenvolver e implementar um processador utilizando uma linguagem de descrição de hardware e a ferramenta Quartus Prime ([INTEL, 2019](#)) e executado em um FPGA. Os conceitos da disciplina teórica foram aplicados em um projeto prático de laboratório, em que um processador funcional baseado em arquitetura ARM foi desenvolvido, incluindo banco de registradores, unidade lógica e aritmética, memória principal e de instruções, juntamente com suas conexões internas. Por fim, foi implementado com sucesso em placa de FPGA de processamento da marca Altera ([TERASIC, 2012](#)).

Palavras-chaves: Arquitetura e Organização de Computadores. ARM. RISC. Processador. Kit FPGA.

Lista de ilustrações

Figura 1 – Simples representação das arquiteturas de Von Neumann e de Harvard	9
Figura 2 – Simples representação das arquiteturas de Von Neumann e de Harvard	11
Figura 3 – Formato de Instruções ARM	13
Figura 4 – Formato de Instruções ARM	15
Figura 5 – Formato de Instruções ARM	16
Figura 6 – Versão 1 do formato de <i>OpCode</i> do tipo Processamento de Dados . . .	17
Figura 7 – Versão final do formato de <i>OpCode</i> do tipo Processamento de Dados .	18
Figura 8 – Formato de <i>OpCode</i> do tipo Registro de Estados	18
Figura 9 – Versão 1 do formato de <i>OpCode</i> do tipo Ramo	19
Figura 10 – Versão final do formato de <i>OpCode</i> do tipo Ramo	19
Figura 11 – Formato de <i>OpCode</i> do tipo Transferência de Dados	19
Figura 12 – Formato de <i>Condition Field</i> e sua condição correspondente	21
Figura 13 – Caminho de dados comum em um MIPS	22
Figura 14 – Caminho de dados projetado	23
Figura 15 – Forma de onda do banco de registradores	30
Figura 16 – Forma de onda do PC	33
Figura 17 – Forma de onda da memória de instruções	38
Figura 18 – Forma de onda do módulo CPSR	39
Figura 19 – Forma de onda da memória de dados (RAM)	46
Figura 20 – Forma de onda da unidade de controle parte 1	48
Figura 21 – Forma de onda da unidade de controle parte 2	49
Figura 22 – Forma de onda da memória de instruções	52
Figura 23 – Código em assembler exemplo - a ser transformado em linguagem de máquina	66
Figura 24 – Código em linguagem de máquina exemplo - transformado a partir do código assembler	66

Lista de tabelas

Tabela 1 – Conjunto de sufixos de suporte	21
---	----

Sumário

1	INTRODUÇÃO	7
2	OBJETIVOS	8
2.1	Geral	8
2.2	Específico	8
3	FUNDAMENTAÇÃO TEÓRICA	9
3.1	Arquitetura de Von Neumann e Harvard	9
3.2	Arquiteturas CISC e RISC	10
3.3	CISC	10
3.4	RISC	10
3.5	Principais diferenças	11
3.6	Arquitetura ARM	11
3.6.1	Funcionamento	12
3.6.2	Sufixos de instruções	12
3.6.3	CPSR	12
3.6.4	Formato de Instruções	13
4	CARACTERÍSTICAS DO PROCESSADOR	14
4.1	Conjunto de Instruções	15
4.1.1	Processamento de Dados	17
4.1.2	Registro de Estados	18
4.1.3	Ramo	19
4.1.4	Transferência de Dados	19
4.2	Conjunto de Sufixos	20
4.2.1	Sufixos de Condição	20
4.2.2	Sufixos de Suporte	21
4.3	Caminho de Dados	22
4.3.1	Unidade de Controle	24
4.3.2	Controlador da Unidade Lógica e Aritmética (<i>ULA Control</i>)	24
4.3.3	Banco de Registradores	25
4.3.4	Memória de Dados (RAM)	25
4.3.5	Memória de Instrução (ROM)	25
4.3.6	Registrador Contador de Programa (PC)	25
4.3.7	Registrador CPSR	26
4.3.8	Flag verifier	26

4.3.9	Flag decoder	26
4.3.10	Multiplexadores	26
5	DESENVOLVIMENTO	28
5.1	Plano de projeto	28
5.2	Linguagem de descrição de máquina	28
5.2.1	Banco de Registradores	29
5.2.2	PC	32
5.2.3	Módulo de saída	34
5.2.4	Memória de instruções	37
5.2.5	Módulo CPSR	38
5.2.5.1	Verificador de <i>Flags</i>	40
5.2.5.2	Decodificador de <i>flags</i>	43
5.2.5.3	CPSR	44
5.2.6	Memória de dados (RAM)	45
5.2.7	Unidade de controle	47
5.2.8	Controlador da ULA	51
5.2.8.1	ULA	53
5.2.9	Divisor de frequência	56
5.2.10	Módulo integrador	57
5.2.11	Módulo final do processador	62
5.3	Assembler	65
5.4	Testes em bancada	67
6	RESULTADOS	68
7	CONCLUSÃO	69
	REFERÊNCIAS	70

1 Introdução

Nos dias atuais, a crescente demanda por dispositivos mais rápidos, eficientes e inteligentes tem impulsionado o avanço das arquiteturas de processadores e a miniaturização dos circuitos integrados. Tecnologias emergentes, como a Internet das Coisas (IoT) e a Inteligência Artificial (IA), requerem processadores cada vez mais especializados e otimizados para desempenhar tarefas específicas, com alto desempenho e baixo consumo de energia. Dentro desse contexto, o desenvolvimento de processadores personalizáveis e configuráveis em dispositivos como FPGAs (Field Programmable Gate Arrays) tem ganhado destaque por sua flexibilidade e capacidade de adaptação a diversas aplicações.

A abordagem prática deste trabalho permitiu a construção de um sistema completo, incluindo banco de registradores, unidade lógica e aritmética (ULA), e memórias principal e de instruções, com o objetivo de conectar os princípios teóricos à realidade tecnológica atual.

Dessa forma, este trabalho demonstra não apenas o domínio dos conceitos acadêmicos, mas também a relevância da aplicação dessas técnicas na criação de soluções para os desafios da computação moderna.

2 Objetivos

2.1 Geral

O objetivo geral deste projeto foi o de projetar um processador funcional e implementá-lo em um *kit FPGA*, partindo de conceitos teóricos adquiridos na disciplina de Arquitetura e Organização de Computadores.

2.2 Específico

Os objetivos específicos da primeira etapa do projeto, presentes neste documento foram:

1. Analisar as diferentes possíveis arquiteturas e organizações de um processador no âmbito da implementação, complexidade, e atualidade.
2. Definir a arquitetura e organização que melhor atenderem as propostas do aluno.
3. Definir o conjunto de instruções e o caminho de instruções, conhecidos na literatura como *instruction set* e *data path*.
4. Desenvolver em linguagem de descrição de máquina (Verilog) módulos que compõem o caminho de instruções previamente detalhado.
5. Integrar todos os módulos em um modelo de processador.
6. Associar ao processador os módulos de entrada e saída para manipulação de entradas e saídas e teste do processador.
7. Apresentar na bancada todos os tópicos anteriores em funcionamento na placa FPGA ([TERASIC](#), 2012).

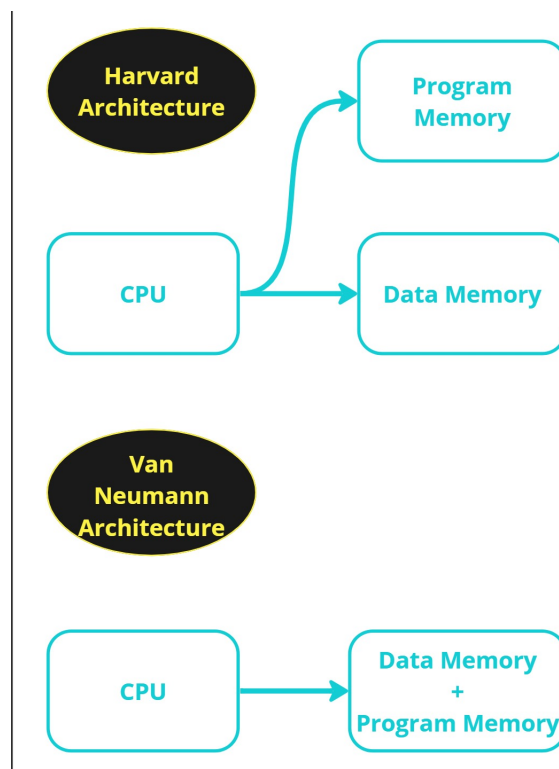
3 Fundamentação Teórica

Existem muitos conceitos gerais que definem o que é um processador e o que é uma arquitetura de computadores. Para compreender as discussões e especificidades da arquitetura descrita como foco e objetivo deste documento, é essencial o conhecimento dos conceitos desta seção, extraídos do livro de Patterson e Hennesy ([HENNESSY JOHN L. PATTERSON, 2014](#)) e algumas outras fontes explicitadas na sequência.

3.1 Arquitetura de Von Neumann e Harvard

A arquitetura de Von Neumann é um modelo de computação que utiliza uma única memória para armazenar tanto as instruções do programa quanto os dados, sendo acessada sequencialmente por uma única unidade de controle. Já a arquitetura de Harvard possui memórias separadas para armazenar instruções e dados, permitindo o acesso simultâneo a ambas, o que pode aumentar a eficiência e a velocidade de processamento em certas situações. ([TMLT, 2021](#))

Figura 1 – Simples representação das arquiteturas de Von Neumann e de Harvard



Fonte: O autor.

3.2 Arquiteturas CISC e RISC

CISC (*Complex Instruction Set Computer*) e RISC (*Reduced Instruction Set Computer*) são dois paradigmas de arquitetura de processadores.

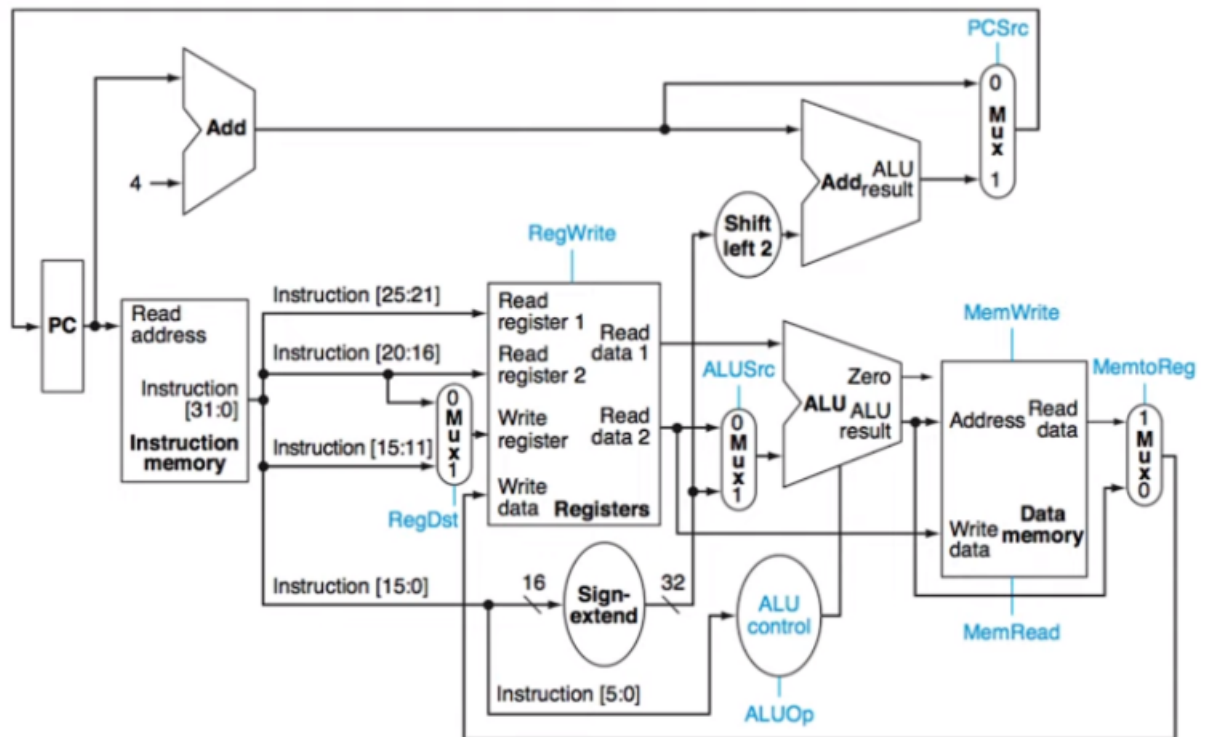
3.3 CISC

Os processadores CISC são projetados com um conjunto complexo de instruções, o que significa que cada instrução pode executar uma série de operações complexas. Essas instruções mais complexas tendem a requerer ciclos de *clock* mais longos para serem executadas. Os processadores CISC geralmente possuem um grande número de modos de endereçamento e um conjunto diversificado de instruções. Exemplos de arquiteturas CISC incluem a família de processadores x86. ([HARRIS, 2016](#))

3.4 RISC

Por outro lado, os processadores RISC são projetados com um conjunto de instruções reduzido e mais simples. Cada instrução em uma arquitetura RISC é projetada para executar uma operação muito simples em um ciclo de *clock*, o que geralmente resulta em tempos de execução mais rápidos. Os processadores RISC tendem a ter menos modos de endereçamento e um conjunto mais limitado de instruções. Exemplos de arquiteturas RISC incluem MIPS, ARM e PowerPC. ([HARRIS, 2016](#))

Figura 2 – Simples representação das arquiteturas de Von Neumann e de Harvard



Fonte: (MOORE, 2015)

3.5 Principais diferenças

Em resumo, enquanto os processadores CISC buscam oferecer um conjunto diversificado e complexo de instruções, os processadores RISC priorizam a simplicidade e a eficiência na execução de instruções.

3.6 Arquitetura ARM

A ideia principal por trás da arquitetura ARM (*Advanced RISC Machine*) é desenvolver uma arquitetura de processador baseada no conceito RISC. A arquitetura ARM foi projetada para oferecer um equilíbrio entre desempenho, consumo de energia e custo, sendo amplamente utilizada em uma variedade de dispositivos, desde smartphones e tablets até dispositivos embarcados e sistemas embarcados de grande escala. A portabilidade, eficiência energética e versatilidade são algumas das características fundamentais da arquitetura ARM.

3.6.1 Funcionamento

Seu funcionamento é, em teoria, similar ao RISC, poucas instruções simples que realizam operações básicas. Porém, a existência de sufixos que podem ser utilizados em todas as instruções fazem com que esse número limitado de instruções seja combinado com sufixos que podem variar em tamanhos e funções, aumentando, desta forma, o número de instruções possíveis de ser aplicadas.

Em suma, enquanto o modelo ARM é uma arquitetura RISC em teoria, a existência de elementos adicionais apêndices ao tradicional *instruction set* fazem com que este modelo se destaque na eficiência, escalabilidade de aplicações e desempenho. (ARM, 2001).

3.6.2 Sufixos de instruções

Os sufixos de instruções são ferramentas da arquitetura ARM que permitem que o operador adicione um sufixo ao código de uma determinada instrução e a torne mais específica, tanto em aplicabilidade quanto em condição. Ou seja, uma instrução acrescida de um sufixo deixa de realizar simplesmente sua função e passa a funcionar de outra forma: o sufixo pode mudar como a instrução é lida, como a instrução é escrita e/ou aplicar uma condição sobre a instrução. No último caso, utiliza-se um registrador extra ao banco de registradores para auxiliar a condição.

3.6.3 CPSR

Na arquitetura ARM, existe um registrador dedicado a salvar um conjunto de estados de um resultado de instrução no momento da execução chamado CPSR (em inglês, literalmente *Current Program Status Register*). O CPSR só salva quando o operador determinar e pode ser independente do banco de registradores. As *flags* são algumas características de estado do resultado e são expressas pelos *bits* N, Z, C e V, que simbolizam, respectivamente:

1. N - Negative: 1 se o resultado for negativo, 0 caso contrário.
2. Z - Zero: 1 se o resultado for zero, 0 caso contrário.
3. C - Carry: 1 se sobrar um carry do resultado, 0 caso contrário.
4. V - Overflow: 1 se o resultado for maior que o tamanho do registrador, 0 caso contrário.

Porém, esta organização de *flags* é a utilizada no documento oficial (ARM7TDMI-S..., 1995) e, por motivos de complexidade e dificuldade de implementação, serão possivelmente reduzidos em implementação futura. (ARM, 2024).

3.6.4 Formato de Instruções

Na ?? é possível verificar os formatos propostos para as instruções de uma arquitetura ARM 7TDMI-S.

Figura 3 – Formato de Instruções ARM

3 3 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 9 8 7 6 5 4 3 2 1 0																																			
1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0																																			
Cond	0	0	I	Opcode				S	Rn				Rd				Operand 2																Data Processing / PSR Transfer		
Cond	0	0	0	0	0	0	A	S	Rd				Rn				Rs	1	0	0	1	Rm				Multiply									
Cond	0	0	0	0	1	U	A	S	RdHi				RdLo				Rn				1	0	0	1	Rm				Multiply Long						
Cond	0	0	0	1	0	B	0	0	Rn				Rd				0	0	0	0	1	0	0	1	Rm				Single Data Swap						
Cond	0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	0	0	0	1	Rn				Branch and Exchange							
Cond	0	0	0	P	U	0	W	L	Rn				Rd				0	0	0	0	1	S	H	1	Rm				Halfword Data Transfer: register offset						
Cond	0	0	0	P	U	1	W	L	Rn				Rd				Offset				1	S	H	1	Offset				Halfword Data Transfer: immediate offset						
Cond	0	1	I	P	U	B	W	L	Rn				Rd				Offset																Single Data Transfer		
Cond	0	1	1																								1								Undefined
Cond	1	0	0	P	U	S	W	L	Rn				Register List																		Block Data Transfer				
Cond	1	0	1	L	Offset																											Branch			
Cond	1	1	0	P	U	N	W	L	Rn				CRd				CP#				Offset								Coprocessor Data Transfer						
Cond	1	1	1	0	CP Opc				CRn				CRd				CP#				CP	0	CRm				Coprocessor Data Operation								
Cond	1	1	1	0	CP Opc				L	CRn				Rd				CP#				CP	1	CRm				Coprocessor Register Transfer							
Cond	1	1	1	1	Ignored by processor																											Software Interrupt			
3 3 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 9 8 7 6 5 4 3 2 1 0																																			
1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0																																			

Figure 4-1: ARM instruction set formats

Fonte: (ARM7TDMI-S..., 1995)

É possível notar e compreender campos muito importantes em todas as instruções. Alguns determinam a instrução, como a existência de um "OpCode" de 4 *bits* e um campo condicional de 4 *bits*. Outros determinam os registradores ou valores que serão operados, como "Rd", "Rn" e "Operand 2".

Alguns desses elementos mais relevantes e que possuem funções essenciais foram mantidos, outros foram adaptados, incorporados às necessidades do projeto e os demais removidos e não serão implementados. Também foi evidenciado um campo "TypeCode" para fins didáticos, que separa as combinações de OpCode em 4 diferentes tipos, ampliando as possibilidades de OpCode em 4 vezes. (ARM, 2001).

4 Características do Processador

O processador será um modelo adaptado da arquitetura ARM presente no *DataSheet* oficial ([ARM7TDMI-S...](#), 1995). As principais diferenças entre ARM e MIPS, funcionalidades e formatos foram mantidos para que o processador projetado seja o mais próximo possível do modelo presente naquele documento.

Para isso, será implementado formato e caminho de instruções diferentes, a serem analisados nas próximas seções.

e teve um bit Link definido em seu *OpCode* (representando duas novas instruções) e o espaço para valor imediato foi drasticamente diminuído, de maneira a se adequar à saída da Unidade Lógica e Aritmética. Isto permitiu que uma instrução do tipo Ramo ocorra utilizando valores de registradores.

O novo formato de instruções adotado no modelo final foi:

Figura 5 – Formato de Instruções ARM

Conjunto de instruções																															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Tipo: Processamento de Dados																															
Cond		0	0	I	S	OpCode				Rd				Rh				Operand 2													
		TypeCode		Immediate		Set Condition		Tabela dedicada				Destination Register				Hit Register				Operand Reg: 0 0 0 0 0											
																				Ro				Blank gap							
																				Operand value:											
																				Immediate operand											
Tipo: Transferência de dados																															
Cond		0	1	I	0	X	X	X	L	Rd				Rh				Operand 2													
		TypeCode		Immediate		Set Condition = 0		Load/Store				Destination Register				Hit Register				Operand Reg: 0 0 0 0 0											
																				Ro				Blank gap							
																				Operand value:											
																				Immediate operand											
Tipo: Ramo																															
Cond		1	1	1	0	L	L'	X	X	X	X	X	X	X	X	X	X	X	X	Offset											
		TypeCode		Immediate = 1		Set Condition = 0		Should go to link				Save Link				Distância pulada a ser incrementada ao registrador PC (Immediate)															

Versão final implementada do formato de instruções. Fonte: O autor.

Os formatos dos *OpCodes* de cada tipo de instrução também foram alterados e podem ser analisados nas subseções a seguir.

4.1.1 Processamento de Dados

Figura 6 – Versão 1 do formato de *OpCode* do tipo Processamento de Dados

Processamento de Dados		
Operação	OpCode	Função
ADD	0 0 0 0	Adicionar
SUB	0 0 0 1	Subtrair
MUL	0 0 1 0	Multiplicar
DIV	0 0 1 1	Dividir (Aux)
AND	0 1 0 0	Porta AND
OR	0 1 0 1	Porta OR
XOR	0 1 1 0	Porta XOR
NOT	0 1 1 1	Porta NOT
MOV	1 0 0 0	Mover reg.s
X	1 0 0 1	INDEFINIDO
X	1 0 1 X	INDEFINIDO
X	1 1 X X	INDEFINIDO

Formato de *OpCode* adaptado e a respectiva instrução a ser realizada do tipo Processamento de Dados. Fonte: O autor.

Figura 7 – Versão final do formato de *OpCode* do tipo Processamento de Dados

Processamento de Dados		
Operação	OpCode	Função
ADD	0 0 0 0	Adicionar
SUB	0 0 0 1	Subtrair
MUL	0 0 1 0	Multiplicar
DIV	0 0 1 1	Dividir (Aux)
AND	0 1 0 0	Porta AND
OR	0 1 0 1	Porta OR
XOR	0 1 1 0	Porta XOR
NOT	0 1 1 1	Porta NOT
MOV	1 0 0 0	Mover reg
IN	1 0 0 1	Input
X	1 0 1 X	INDEFINIDO
X	1 1 X X	INDEFINIDO

Formato de *OpCode* final modificado do planejamento inicial. Fonte: O autor.

4.1.2 Registro de Estados

Este tipo de instrução possui apenas uma versão de formato pois deixou de existir no final do projeto.

Figura 8 – Formato de *OpCode* do tipo Registro de Estados

Registro de Estados		
Operação	OpCode	Função
TST	0 0 0 0	Porta AND
TEQ	0 0 0 1	Teste iguais
CMP	0 0 1 0	Subtrair
NEG	0 0 1 1	Somar
X	0 1 X X	INDEFINIDO
X	1 X X X	INDEFINIDO

Formato de *OpCode* adaptado e a respectiva instrução a ser realizada do tipo Registro de Estados. Fonte: O autor.

4.1.3 Ramo

Figura 9 – Versão 1 do formato de *OpCode* do tipo Ramo

Ramo		
Operação	OpCode	Função
B	1 X X X	Branch
BL	0 X X X	Branch Link

Formato de *OpCode* final modificado do planejamento inicial. Fonte: O autor.

Figura 10 – Versão final do formato de *OpCode* do tipo Ramo

Ramo		
Operação	OpCode	Função
B	0 0 X X	Branch
BL	1 0 X X	Branch Link
Link	0 1 X X	Link
LL*	1 1 X X	Link and Link

Formato de *OpCode* adaptado e a respectiva instrução a ser realizada do tipo Ramo. Fonte: O autor.

4.1.4 Transferência de Dados

Figura 11 – Formato de *OpCode* do tipo Transferência de Dados

Transferência de Dados		
Operação	OpCode	Função
Load	X X X 1	L. Rh in Rd
Store	X X X 0	S. Op2 in Rh

Formato de *OpCode* adaptado e a respectiva instrução a ser realizada do tipo Ramo. Fonte: O autor.

A partir de [Figura 4](#) e ?? também é possível notar que as instruções de transferência de dados funcionam pela ativação de *bit a bit* do *OpCode*, cada um com um significado

diferente. O *bit* U indica se o valor imediato de endereçamento por deslocamento é um deslocamento para cima (*Up*, sinal alto) ou para baixo (*Down*, sinal baixo); o *bit* B diz se este deslocamento é realizado de 4 em 4 ou de 8 em 8 *bits*; e o bit L diz se a operação é de *Load* (carregar) ou *Store* (guardar).

Apesar das diferentes possibilidades de bits, apenas o bit L foi implementado.

4.2 Conjunto de Sufixos

As instruções ainda contam com configurações adicionais representadas por sufixos, em que a presença de um sufixo ou outro pode alterar o comportamento de um valor na operação, impedir que a instrução ocorra ou alterar o CPSR.

O comportamento desses sufixos e como eles são definidos dependem dos dois tipos: Sufixos de Condição e Sufixos de Suporte.

4.2.1 Sufixos de Condição

No CPSR original (de 32 *bits*) ([ARM, 2024](#)), o bit 31 é o N, o 30 é o Z, o 29 é o C e o 28 é o V. Esses *bits* podem ser verificados a cada instrução, por meio dos *bits* 31 a 28 de cada instrução, chamados de campo de condição (*Condition Field*). Em outras palavras, todas as instruções são condicionais, determinadas pelos bits do CPSR e a condição da instrução varia em função do *Condition Field*.

Para o programador, essas condições são definidas por meio de um sufixo específico para cada condição. Este processador apresentou um CPSR de 4 bits, mas apenas duas flags foram utilizadas, deixando espaço para futuras implementações. Os sufixos e os códigos podem ser interpretados da seguinte tabela:

Figura 12 – Formato de *Condition Field* e sua condição correspondente

Flag Verifier		
Condição	Código	Função
Always	0 0 0 0	Operar
EQ	0 0 0 1	Z1
NEQ	0 0 1 0	Z0
GT	0 0 1 1	Z0 e N0
GTEQ	0 1 0 0	Z1 ou N0
LT	0 1 0 1	N1
LTEQ	0 1 1 0	Z1 ou N1
NOT	0 1 1 1	Porta NOT
X	1 X X X	INDEFINIDO

Na imagem, a coluna "Função" mostra quais *flags* são verificadas pelo módulo *Flag Verifier* e em que nível lógico estão, no formato [bit][nível-lógico]. Por exemplo, "Z1" representa *flag* Z em nível alto. Fonte: O autor.

4.2.2 Sufixos de Suporte

Os Sufixos de Suporte são aqueles que não influenciam diretamente na execução da instrução, mas alteram o funcionamento dela. Primeiramente, o *bit* I explicita se o segundo operando será considerado um valor em binário (até 10 *bits*) ou se será considerado um endereço de registrador que guarda um valor em binário (neste caso, podendo chegar até o tamanho dos registradores, 32).

Tabela 1 – Conjunto de sufixos de suporte

Sufixo	Função	Operação	Bit
S	Set CPSR	Atualiza <i>flags</i> baseado no resultado da Unidade Lógica e Aritmética	24
I	Imediato	Operando 2 é definido como imediato	25

Na coluna "Operação" está a funcionalidade que é alterada ou acrescida à operação caso o sufixo esteja presente. Se está presente, o correspondente *bit* deve estar em nível alto na instrução. Fonte: O Autor

caminhos extras para contemplar a instrução de link e as conexões com os módulos de entrada e saída.

O design do *datapath* final, porém, é muito grande para caber neste documento. Portanto, o autor deste projeto o deixou disponível em hbreis.pythonanywhere.com para que todos possam visualizá-lo e consultá-lo (REIS, 2024b).

Neste link, também é possível observar, logo abaixo do *datapath*, um fluxograma de ciclo de clock, que foi necessário para compreender melhor a dinâmica de atualizações e leituras de processadores e memórias e realização de operações durante as instruções.

4.3.1 Unidade de Controle

Adicionado apenas nas últimas versões do modelo, a Unidade de Controle (*Control Unit, CU*) serviu como centro de distribuição de códigos de instruções e alguns endereços de registradores. No *datapath* final, é possível compreender exatamente quais sinais entram e saem de seu interior. Em suma, a UC é uma grande central de distribuição e funciona como um gestor de tarefas, distribuindo pedaços das instruções e outras especificações codificadas, alteradas a cada instrução.

4.3.2 Controlador da Unidade Lógica e Aritmética (*ULA Control*)

1. ULA

O funcionamento da ULA inicial dependia do Tipo de Processamento de Dados e Registro de Dados e o *OpCode* define suas operações. Porém, ao final, a ULA funcionava em todos os tipos de instrução.

As operações possíveis que a ULA realiza no tipo Processamento de Dados são adição (*AND*), subtração (*SUB*), multiplicação (*MUL*), divisão (*DIV*), porta *AND*, porta *OR*, porta *XOR* e porta *NOT*, movimento de valores para um registrador e entrada periférica, conforme descrito na Figura 7.

As operações "teste", "teste iguais", "comparar" e "negativo" descritas na Figura 8 (equivalentes às operações *AND*, *SUB*, *SUB* e *ADD* respectivamente) foram descartadas.

O tipo Transferência de Dados também utiliza a ULA como intermediário. Neste caso, a ULA envia o operando 2 para instruções de *Store* (salvamento em RAM), permitindo guardar um valor de registrador ou de imediato na memória de dados.

O tipo *Branch* não utilizava a ULA inicialmente. Porém, após as mudanças no *design*, a ULA passou a enviar o valor do operando 2 para o PC como valor de *branch*, permitindo a manipulação entre valor imediato e valor de registrador para esse tipo de instrução.

2. Controlador

O controlador é responsável por harmonizar as informações que entram e saem da ULA, ou seja, o controlador pré-processa os valores, enviando apenas os valores que serão operados para dentro da ULA e os que não serão para os próximos módulos.

A ULA está contida dentro do controlador, de forma que apenas ele tenha acesso a suas entradas e saídas e possa distribuí-las da melhor forma possível.

4.3.3 Banco de Registradores

O banco de registradores apresenta todos os 32 registradores de 32 *bits* que serão utilizados pelo programa. Ele recebe os endereços da Unidade de Controle, que vêm da instrução atual.

Lê os dados imediatamente, porém só salva dados na subida do *clock*, imediatamente antes da mudança de instrução, para que todas as operações possam ser realizadas antes do salvamento.

4.3.4 Memória de Dados (RAM)

A memória de dados é um armazenamento maior da memória que será utilizada pelo programa, porém de acesso mais lento. Pode ser acessada por operações do tipo transferência de dados, como descrito em 4.1.

Salva na subida do *clock*, assim como o banco de registradores, mas, diferentemente, lê apenas na descida para evitar problemas de leitura antes que a escrita anterior tenha sido finalizada.

4.3.5 Memória de Instrução (ROM)

A memória de instrução é um banco de registradores dedicado exclusivamente a guardar as instruções do programa que será executado em ordem. Ela será acessada utilizando o PC como "argumento-índice".

É o único registrador que é lido na subida do *clock*, pois ele determina o início da próxima instrução e não pode ser lido antes que todos os módulos tenham finalizado suas funções. Seu endereço de acesso vem do PC e as instruções são carregadas na compilação por meio de um arquivo de texto ".txt".

4.3.6 Registrador Contador de Programa (PC)

O *Program Counter* é, como o próprio nome já diz, o contador do programa, é ele que diz à memória de instrução qual instrução está sendo realizada no momento. Ele

pode ser alterado por operações do tipo Ramo para que efetue um salto (progressor ou regressor em relação à ordem de execução), ferramenta essencial para o funcionamento de computadores atuais.

Este registrador é atualizado a cada descida de *clock* e enviado diretamente à ROM, que só é lida na próxima subida de *clock*.

Os módulos *PC Adder* e *Branch Adder* foram simplificados para caber dentro deste módulo, mas suas funções estão presentes internamente.

4.3.7 Registrador CPSR

O registrador CPSR é um elemento importante da arquitetura ARM, uma vez que é essencial para o funcionamento o campo condicional nas instruções, que está presente em todas elas. Ou seja, todas as instruções que não apresentem comportamento *ALWAYS* farão uma verificação a este registrador.

Além disso, é nele que se localizam as *flags* de estado, que indicam os estados de uma operação antiga guardada e é isto o que o torna essencial.

Para atualizar essas *flags* do CPSR, basta utilizar as instruções de registro de estados ([Figura 8](#)) ou então o sufixo S do conjunto de sufixos (??) para que as *flags* do resultado sejam atualizadas.

O CPSR é atualizado no ciclo de subida do *clock*.

4.3.8 Flag verifier

Flag verifier é responsável por comparar a condição da instrução definida pelo sufixo dela ([subseção 4.2.1](#)) com o estado do CPSR e diz se a operação será realizada ou não.

4.3.9 Flag decoder

Flag decoder é responsável por decodificar o resultado das operações e enviar este resultado para o CPSR. Em outras palavras, ele interpreta as características ([subseção 3.6.3](#)) do resultado para salvar no CPSR.

4.3.10 Multiplexadores

Os multiplexadores presentes na [Figura 15](#) são simples verificações que permitem o tipo correto de dado passar adiante no caminho. Ele seleciona, com base na instrução e nos sufixos, se o dado que prosseguirá é um imediato, um endereço, se o dado será escrito

ou não ou se o PC será atualizado em um valor maior que o de uma instrução (instruções de ramo na [Figura 12](#)).

5 Desenvolvimento

Neste capítulo estão dispostos todos os passos realizados até a entrega final do processador, desde a criação do plano de projeto, passando pelo teste em forma de onda, até o teste em bancada.

5.1 Plano de projeto

Todas as seções anteriores foram fruto do plano de projeto inicial, que, apesar de sofrer alterações, se demonstrou sólido como base para o desenvolvimento geral e a organização em esquemático, tabelas e anotações teóricas foram essenciais para que os estágios intermediários não se distraíssem do seu propósito geral, de desenvolver um processador de arquitetura baseada em ARM.

O plano de projeto não apresentou grandes dificuldades, exceto a falta de material de estudo sobre arquitetura ARM. Esta etapa demandou, porém, uma quantidade grande de tempo investido para que a lógica por trás do processador não fosse prejudicada.

5.2 Linguagem de descrição de máquina

Esta etapa é a primeira da aplicação prática do projeto. Nela, iniciou-se a programação em linguagem de descrição de máquina. Para isso e para compilar o código, foi utilizado a ferramenta Quartus Prime versão 19.1 (INTEL, 2019).

Esta seção é a mais extensa do relatório. Isso reflete também ao grau de dificuldade e quantidade de tempo investido nesta fase. Com margem altíssima para erros e complexas interações entre os modos, qualquer deslize pode causar e, em algumas situações causou, erros que impediram completamente o funcionamento do processador.

A adição de algumas funcionalidades de última hora, como *Branch and Link* e *input* também foram uma grande barreira que dificultou o progresso nas etapas finais de desenvolvimento, porque alterar o processador após finalizado pode tornar alguns módulos disfuncionais por acaso e demanda tempo e alto conhecimento sobre a máquina desenvolvida.

O código e sua evolução podem ser analisados mais cuidadosamente em <https://github.com/HeitorReis/ARM_processor/settings> (REIS, 2024a). As próximas subseções explicitam os módulos e seus códigos finais.

5.2.1 Banco de Registradores

O módulo registerBank implementa um banco de registradores usado em uma arquitetura de processador. Ele é composto por 32 registradores de uso geral, cada um com 32 bits, além de um registrador de link adicional para armazenar endereços de retorno durante chamadas de função ou operações de desvio. O módulo interage com um sinal de clock e vários sinais de entrada e saída para leitura e escrita de dados.

1. Entradas:

- a) RhAddress, RoAddress, RdAddress: Endereços de 5 bits usados para selecionar os registradores fonte e destino.
- b) WriteData: Dados de 32 bits que serão escritos no registrador de destino (Rd).
- c) new_LinkValue: Valor de 32 bits que será armazenado no registrador de link.
- d) Load, TypeCode, write_condition, should_store_link: Sinais de controle que determinam quando os dados serão escritos no banco de registradores ou no registrador de link.
- e) clock: Usado para acionar operações síncronas.

2. Saídas:

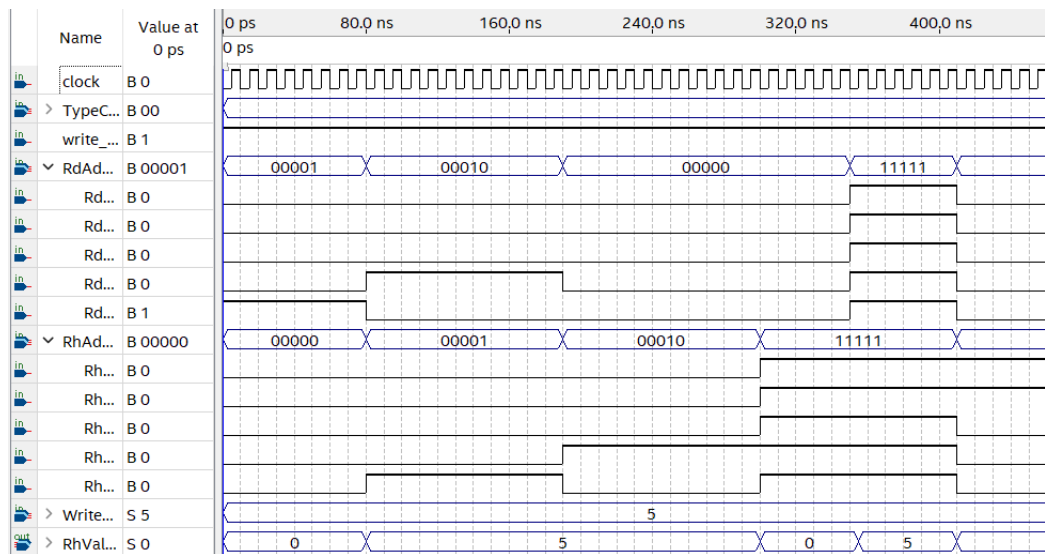
- a) RhValue, RoValue, RdValue: Saídas de 32 bits que correspondem aos valores nos registradores endereçados por Rh, Ro e Rd, respectivamente.
- b) LinkValue: Valor atual armazenado no registrador de link.
- c) r0_value, r31_value: Para fins de teste, essas saídas representam os valores dos registradores r0 e r31.

3. Funcionalidade:

- a) Na borda de subida do clock, se o sinal write_condition estiver ativo, o módulo grava os dados no registrador de destino (Rd) com base nos sinais de controle TypeCode e Load. Se o sinal should_store_link estiver ativo, o módulo atualiza o registrador de link com o valor de new_LinkValue.
- b) O módulo também fornece continuamente os valores dos registradores fonte (Rh e Ro), do registrador de destino (Rd) e do registrador de link para uso posterior.

Para testar se o módulo funciona corretamente, foi utilizado a seguinte forma de onda:

Figura 15 – Forma de onda do banco de registradores



Fonte: O autor.

O código final do banco de registradores implementado foi:

```
module registerBank (

    // Addresses
    input [4:0] RhAddress,
    input [4:0] RoAddress,
    input [4:0] RdAddress,

    // Values
    input [31:0] WriteData,
    input [31:0] new_LinkValue,

    // Codes
    input Load,
    input [1:0] TypeCode,
    input write_condition,
    input should_store_link,

    // Clock
    input clock,

    // Outputs
    output [31:0] RhValue,
```

```
output [31:0] RoValue,
output [31:0] RdValue,
output [31:0] LinkValue,

// Test
output [31:0] r0_value,
output [31:0] r31_value
);

reg [31:0] regBank [31:0]; // Register bank
reg [31:0] LinkRegister;

always@(posedge clock) begin
    // If should write
    if (write_condition) begin

        if ( (TypeCode == 2'b00) | ( (TypeCode == 2'b01) & Load ) )
            regBank[RdAddress] = WriteData; // Write

        if ( should_store_link )
            LinkRegister = new_LinkValue;
    end
end

assign r31_value = regBank[31];
assign r0_value = regBank[0];

assign RhValue = regBank[RhAddress];
assign RoValue = regBank[RoAddress];
assign RdValue = regBank[RdAddress];

assign LinkValue = LinkRegister;

endmodule
```

5.2.2 PC

O módulo PC_main implementa o contador de programa (PC) de um processador, que controla a sequência de execução das instruções. Ele recebe sinais de controle para definir o próximo endereço de instrução, possibilitando saltos condicionais e o uso de um valor armazenado no registrador de link, como em chamadas de função ou interrupções. O comportamento do contador é sincronizado com o sinal de clock e pode ser temporariamente interrompido por um sinal de pausa.

1. Entradas:

- a) `branch_value`: Valor de desvio utilizado em saltos (branch).
- b) `link_value`: Valor do registrador de link, usado em saltos para endereços de retorno.
- c) `should_branch`, `write_condition`, `should_branch_to_link`: Sinais de controle que determinam quando e como o salto será realizado.
- d) `halt_temporarily_signal`: Sinal que indica se o contador de programa deve ser temporariamente pausado.
- e) `clock`: Sinal de clock que sincroniza o incremento e as operações de salto do contador de programa.
- f) `reset`: Sinal de reinicialização, que define o valor do contador de programa para zero.

2. Saída:

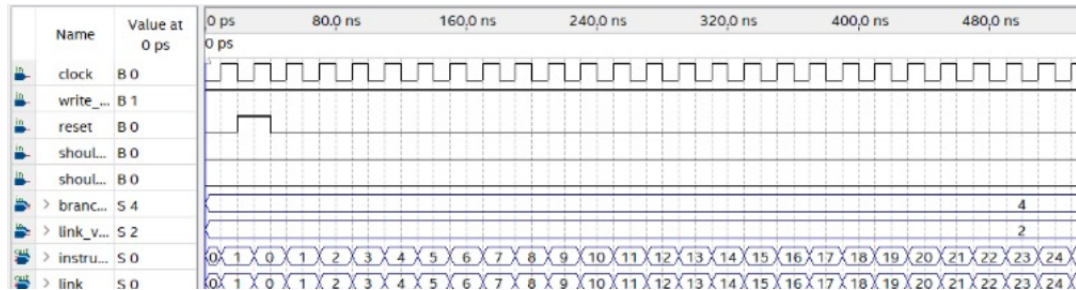
- a) `instruction_address`: Endereço da instrução atual, ou seja, o valor do contador de programa.

3. Funcionalidade:

- a) O contador de programa (`instruction_address`) é inicializado em zero. A cada borda negativa do clock, ele incrementa o valor de `instruction_address`, salvo se o sinal de pausa (`halt_temporarily_signal`) estiver ativo.
- b) Se o sinal `write_condition` for ativado, o módulo verifica se deve realizar um salto (`should_branch`). Caso o salto deva ocorrer, ele determina se deve saltar para o valor armazenado no registrador de link (`should_branch_to_link`) ou adicionar o valor de desvio (`branch_value`) ao endereço atual.
- c) Se o sinal de reinicialização (`reset`) for ativado, o contador de programa é resetado para zero.

Para testar se o módulo funciona corretamente, foi utilizado a seguinte forma de onda:

Figura 16 – Forma de onda do PC



Fonte: O autor.

O código final do PC implementado foi:

```
module PC_main (

    input [31:0] branch_value,
    input [31:0] link_value,

    input should_branch,
    input write_condition,
    input should_branch_to_link,

    input halt_temporarily_signal,
    input clock,
    input reset,

    output reg signed [31:0] instruction_address
);

initial begin
    instruction_address <= 5'b0;
end

always@( negedge clock ) begin // When clock ticksdown
    if (~halt_temporarily_signal) begin
        if (reset) // If reset
            instruction_address = 0; // Reset
    end
end
```

```
else begin // Else
    instruction_address = instruction_address + 1; // Add one to PC

    if (write_condition) begin // If should save
        if (should_branch) begin // If should branch
            if (should_branch_to_link) // If should branch to link

                instruction_address = link_value; // Then, branch to link

            else
                instruction_address = instruction_address + branch_value;
                // Else, branch to value

        end end end end end

    assign link = instruction_address;

endmodule
```

5.2.3 Módulo de saída

O módulo `output_module` é responsável por converter um número de 32 bits em uma representação decimal de quatro dígitos e exibi-lo em quatro displays de 7 segmentos. Para isso, ele divide o número em seus dígitos individuais e os mapeia para os segmentos correspondentes de cada display.

1. Entradas:

- a) number: Número de 32 bits que será exibido.

2. Saídas:

- a) HEX0, HEX1, HEX2, HEX3: Cada um é um vetor de 7 bits que controla os segmentos de um display de 7 segmentos.

3. Funcionalidade:

- a) O número de 32 bits é dividido em quatro partes: `number0`, `number1`, `number2` e `number3`. Cada uma dessas partes contém um dígito do número, representado em forma decimal. O dígito mais significativo (milhares) é armazenado em `number3`, enquanto o dígito menos significativo (unidades) é armazenado em `number0`.

- b) Cada um dos dígitos é então passado para o módulo `to_display`, que converte o valor binário do dígito (de 0 a 9) na configuração apropriada de segmentos para exibição no display de 7 segmentos.

O módulo `to_display` é responsável por mapear um número binário de 4 bits para a correspondente configuração de segmentos em um display de 7 segmentos.

1. Entradas:

- a) `number`: Um número de 4 bits que representa um dígito (de 0 a 9).

2. Saídas:

- a) `display`: Um vetor de 7 bits que define quais segmentos do display serão acesos para representar o número.

3. Funcionalidade:

- a) O número de entrada é comparado com cada dígito decimal (de 0 a 9) e o código correspondente para o display de 7 segmentos é gerado. Para números maiores que 9, o módulo subtrai 10, tratando-os de forma adequada.
- b) O caso `case` é usado para associar cada número de 0 a 9 com a configuração de segmentos correta. Se o número for inválido, todos os segmentos serão apagados.

O código final do módulo de saída implementado foi:

```
module output_module(  
    input  [31:0]number,  
  
    output [6:0]HEX0,  
    output [6:0]HEX1,  
    output [6:0]HEX2,  
    output [6:0]HEX3  
);  
  
    wire [3:0] number0;  
    wire [3:0] number1;  
    wire [3:0] number2;  
    wire [3:0] number3;  
  
    assign number0 = number;  
    assign number1 = number/10;
```

```
assign number2 = number/100;
assign number3 = number/1000;

to_display hex0(
    .number(number0),
    .display(HEX0)
);
to_display hex1(
    .number(number1),
    .display(HEX1)
);
to_display hex2(
    .number(number2),
    .display(HEX2)
);
to_display hex3(
    .number(number3),
    .display(HEX3)
);
endmodule

module to_display(
    input [3:0] number,
    output reg [6:0] display
);
    reg [3:0] digit;

    always@* begin
        if (number > 9)
            digit <= number - 10;

        else
            digit <= number;

        case(digit)
            4'd0: display = 7'b1000000;
            4'd1: display = 7'b1111001;
            4'd2: display = 7'b0100100;
            4'd3: display = 7'b0110000;
```

```
4'd4: display = 7'b0011001;
4'd5: display = 7'b0010010;
4'd6: display = 7'b0000010;
4'd7: display = 7'b1111000;
4'd8: display = 7'b0000000;
4'd9: display = 7'b0011000;
default: display = 7'b1111111;
endcase end
endmodule
```

5.2.4 Memória de instruções

O módulo `InstructionMemory` implementa uma memória de instruções em um processador. Ele utiliza uma memória ROM (Read-Only Memory) que armazena as instruções do programa a serem executadas. O conteúdo dessa memória é carregado de um arquivo externo e é acessado de acordo com o endereço fornecido. Este módulo é parametrizável em termos de largura de dados e de endereços, o que oferece flexibilidade no design.

1. Entradas:

- a) `addr`: Endereço de entrada que indica a posição da memória ROM de onde será lida a instrução. Sua largura é definida pelo parâmetro `ADDR_WIDTH`.
- b) `clock`: Sinal de clock que sincroniza a leitura da memória de instruções.

2. Saídas:

- a) `instruction`: Instrução de 32 bits lida da ROM, que será utilizada pelo processador.
- b) `first_instr_line_test`: Saída adicional para fins de teste que pode armazenar a primeira instrução carregada.

3. Funcionalidade:

- a) A memória de instruções é inicializada durante a simulação com o conteúdo de um arquivo externo (`single_port_rom_init.txt`) utilizando o comando `$readmemb`. O arquivo contém os dados que representam as instruções do programa a serem executadas.
- b) Em cada borda de subida do sinal de clock, o módulo lê a instrução armazenada no endereço fornecido (`addr`) e a coloca na saída `instruction`.

- a) ALUresult: Resultado de 32 bits gerado pela unidade lógica e aritmética (ALU).
- b) set_cond_bit: Sinal que indica se os bits de condição devem ser atualizados.
- c) zero: Sinal que indica se o resultado da ALU é zero.
- d) negative: Sinal que indica se o resultado da ALU é negativo.
- e) Cond_field: Campo de condição de 4 bits que define as condições sob as quais a instrução deve ser executada.
- f) clock: Sinal de clock que sincroniza as atualizações do registro CPSR.

2. Saídas:

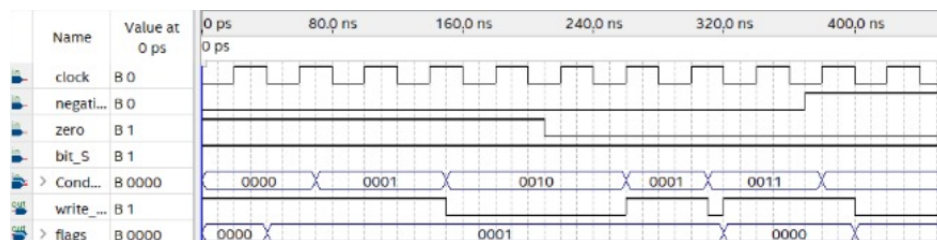
- a) write_condition: Sinal que indica se as flags devem ser atualizadas.
- b) current_flags: Flags de status atuais (para fins de teste).

3. Funcionalidade:

- a) O módulo utiliza três sub-módulos principais:
 - i. FlagVerifier: Verifica as condições definidas pelo campo Cond_field e decide se as flags devem ser atualizadas com base nas flags atuais.
 - ii. CPSRegister: Atualiza o registro CPSR com novos estados das flags com base no sinal de clock e na condição de escrita.
 - iii. FlagDecoder: Decodifica o resultado da ALU e os sinais zero e negativo para determinar os novos estados das flags.

Para testar se o módulo funciona corretamente, foi utilizado a seguinte forma de onda:

Figura 18 – Forma de onda do módulo CPSR



Fonte: O autor.

O código final do módulo CPSR implementado foi:

```
module CPSR_module (
  input [31:0] ALUresult,
  input set_cond_bit,
```

```

input zero,
input negative,
input [3:0] Cond_field,
input clock,

output write_condition,
output [3:0] current_flags // Test
);
wire [3:0] currrent_flags;
wire [3:0] new_flags;

FlagVerifier FlagVerifier(
    .cond_field(Cond_field),
    .flags(currrent_flags),
    .write_condition(write_condition)
);
CPSRegister CPSRegister(
    // Set Condition code
    .should_set(set_cond_bit),
    .write_condition(write_condition),
    // CPSR bits to be updated
    .new_states(new_flags),
    .clock(clock),
    .current_states_reg(currrent_flags)
);
FlagDecoder FlagDecoder(
    .result_in(ALUresult),
    .zero(zero),
    .negative(negative),
    .new_states(new_flags)
);
endmodule

```

5.2.5.1 Verificador de *Flags*

O módulo FlagVerifier é responsável por verificar as condições definidas pelo campo `cond_field` e decidir se a operação deve prosseguir com base nos flags de status atuais. Ele determina se a condição para a execução de uma instrução é atendida, configurando o sinal `write_condition` adequadamente.

1. Entradas:

- a) `cond_field`: Campo de condição de 4 bits que define o tipo de comparação a ser realizada.
- b) `flags`: Flags de status atuais de 4 bits que são usadas para a comparação.

2. Saída:

- a) `write_condition`: Sinal que indica se a condição para a execução da instrução é satisfeita.

3. Funcionalidade:

- a) O módulo utiliza um bloco `case` para verificar o campo `cond_field` e comparar os flags com base na condição especificada.
 - i. 4'b0000: Sempre (ALWAYS). O sinal `write_condition` é definido como 1, independentemente dos flags.
 - ii. 4'b0001: Igual (EQUAL). O sinal `write_condition` é 1 se o flag de zero (`flags[0]`) estiver ativo.
 - iii. 4'b0010: Não Igual (NOT EQUAL). O sinal `write_condition` é 1 se o flag de zero (`flags[0]`) não estiver ativo.
 - iv. 4'b0011: Maior que (GREATER THAN). O sinal `write_condition` é 1 se o flag de zero (`flags[0]`) e o flag negativo (`flags[1]`) não estiverem ativos.
 - v. 4'b0100: Maior ou Igual (GREATER THAN OR EQUAL). O sinal `write_condition` é 1 se o flag de zero (`flags[0]`) estiver ativo ou o flag negativo (`flags[1]`) não estiver ativo.
 - vi. 4'b0101: Menor que (LESS THAN). O sinal `write_condition` é 1 se o flag negativo (`flags[1]`) estiver ativo.
 - vii. 4'b0110: Menor ou Igual (LESS THAN OR EQUAL). O sinal `write_condition` é 1 se o flag de zero (`flags[0]`) ou o flag negativo (`flags[1]`) estiverem ativos.
 - viii. Default: Se o campo `cond_field` não corresponder a nenhum dos casos acima, o sinal `write_condition` é definido como 0.

O código final do verificador de *flags* implementado foi:

```
module FlagVerifier (  
  
    input [3:0] cond_field,  
    input [3:0] flags,
```

```
output reg write_condition

);
always@* begin
  case (cond_field)
    4'b0000: write_condition = 1; // ALWAYS case
    4'b0001: begin // EQUAL case
      if (flags[0])
        write_condition = 1;
      else
        write_condition = 0;
    end
    4'b0010: begin // NOT EQUAL case
      if (~flags[0])
        write_condition = 1;
      else
        write_condition = 0;
    end
    4'b0011: begin // GREATER THAN case
      if (~flags[0] & ~flags[1])
        write_condition = 1;
      else
        write_condition = 0;
    end
    4'b0100: begin // GREATER THAN OR EQUAL case
      if (flags[0] | ~flags[1])
        write_condition = 1;
      else
        write_condition = 0;
    end
    4'b0101: begin // LESS THAN case
      if (flags[1])
        write_condition = 1;
      else
        write_condition = 0;
    end
    4'b0110: begin // LESS THAN OR EQUAL case
      if (flags[0] | flags[1])
        write_condition = 1;
    end
  endcase
end
```

```

        else
            write_condition = 0;
        end
        default: write_condition = 0;
    endcase end
endmodule

```

5.2.5.2 Decodificador de *flags*

O módulo FlagDecoder é responsável por decodificar o resultado da ALU e os sinais de status (zero e negativo) para gerar os novos estados das flags.

1. Entradas:

- a) `result_in`: Resultado de 32 bits gerado pela unidade lógica e aritmética (ALU). Este valor não é utilizado diretamente no cálculo das flags neste módulo.
- b) `zero`: Sinal que indica se o resultado da ALU é zero.
- c) `negative`: Sinal que indica se o resultado da ALU é negativo.

2. Saída:

- a) `new_states`: Vetor de 4 bits representando os novos estados das flags. O bit 3 e 2 são definidos como 00, o bit 0 é definido pelo sinal zero, e o bit 1 é definido pelo sinal negativo.

3. Funcionalidade:

- a) O módulo atribui os valores aos bits de `new_states` da seguinte forma:
 - i. Bits 3 e 2 são sempre definidos como 00.
 - ii. Bit 0 é definido pelo sinal zero (`zero`).
 - iii. Bit 1 é definido pelo sinal negativo (`negative`).

O código final da memória de instruções implementado foi:

```

module FlagDecoder (
    input [31:0] result_in,
    input zero,
    input negative,

    output [3:0] new_states
);

```

```

assign new_states[3:2] = 2'b00;
assign new_states[0] = zero;
assign new_states[1] = negative;
endmodule

```

5.2.5.3 CPSR

O módulo CPSRegister é responsável por atualizar o registro CPSR (Current Program Status Register) com base em sinais de controle e novos estados das flags.

1. Entradas:

- a) `should_set`: Sinal que indica se o registro CPSR deve ser atualizado.
- b) `write_condition`: Sinal que determina se as condições para atualização do registro CPSR são atendidas.
- c) `new_states`: Vetor de 4 bits que representa os novos estados das flags a serem carregados no registro CPSR.
- d) `clock`: Sinal de clock que sincroniza a atualização do registro CPSR.

2. Saída:

- a) `current_states_reg`: Registro de 4 bits que mantém o estado atual das flags.

3. Funcionalidade:

- a) O registro `current_states_reg` é inicializado com o valor 4'b0000.
- b) Em cada borda de descida do sinal de clock, o módulo verifica se `should_set` e `write_condition` estão ativos.
 - i. Se ambos os sinais estiverem ativos, o registro `current_states_reg` é atualizado com o valor de `new_states`.

O código final da memória de instruções implementado foi:

```

module CPSRegister (
  // Set Condition code
  input should_set,
  input write_condition,
  // CPSR bits to be updated
  input [3:0] new_states,
  input clock,
  output reg [3:0] current_states_reg

```

```
);  
initial  
    current_states_reg = 4'b0000;  
always@( negedge clock )  
    if (should_set & write_condition)  
        current_states_reg = new_states;  
endmodule
```

5.2.6 Memória de dados (RAM)

O módulo DataMemory é responsável por gerenciar a memória de dados em um processador. Ele realiza operações de leitura e escrita na RAM com base em sinais de controle e endereços fornecidos.

1. Entradas:

- a) `new_data`: Dados de 32 bits a serem escritos na memória.
- b) `mem_addr`: Endereço de 32 bits na memória onde os dados serão lidos ou escritos.
- c) `Load`: Sinal que indica se a operação é de leitura.
- d) `clock`: Sinal de clock que sincroniza as operações de leitura e escrita.
- e) `write_condition`: Sinal que determina se a operação de leitura ou escrita deve ser realizada.
- f) `TypeCode`: Código de 2 bits que especifica o tipo de operação a ser realizada.

2. Saídas:

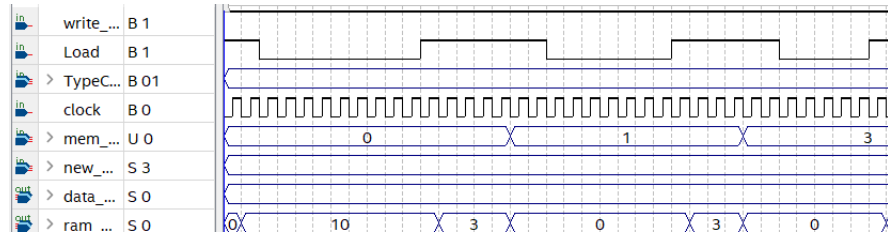
- a) `ram_output`: Dados de 32 bits lidos da memória.
- b) `data_saved`: Dados de 32 bits que foram salvos na memória.

3. Funcionalidade:

- a) O módulo utiliza dois blocos sempre (**always**) para ler e escrever na memória RAM.
 - i. Em cada borda de subida do sinal de clock (**posedge clock**), se **write_condition** estiver ativo e **Load** estiver inativo e **TypeCode** for 2'b01, os dados de **new_data** são escritos na memória no endereço especificado por **mem_addr**.
 - ii. Em cada borda de descida do sinal de clock (**negedge clock**), se **write_condition** estiver ativo e **Load** estiver ativo e **TypeCode** for 2'b01, os dados são lidos da memória no endereço especificado por **mem_addr** e atribuídos a **ram_output**.

Para testar se o módulo funciona corretamente, foi utilizado a seguinte forma de onda:

Figura 19 – Forma de onda da memória de dados (RAM)



Fonte: O autor.

O código final da memória de dados (RAM) implementado foi:

```
module DataMemory
(
    input [31:0] new_data,
    input [31:0] mem_addr,
    input Load, clock, write_condition,
    input [1:0]TypeCode,

    output reg [31:0] ram_output,
    output reg [31:0] data_saved
);
    // Declare the RAM variable
    reg [31:0] ram [31:0];

    always @ (posedge clock)begin // Write in positive edge
        if (write_condition) begin
            // If Store -> WRITE
            if (~Load & (TypeCode == 2'b01))
                ram[mem_addr] <= new_data;
        end end

    always @ (negedge clock) begin // Read in negative edge
        if (write_condition) begin
            // If Load -> READ
            if (Load & (TypeCode == 2'b01))
                ram_output <= ram[mem_addr];
        end end
```

endmodule

5.2.7 Unidade de controle

O módulo ControlUnit é responsável por interpretar a instrução fornecida e gerar os sinais de controle necessários para gerenciar diversos componentes do processador, incluindo o banco de registradores, a unidade aritmética e lógica (ALU), a memória de dados, o módulo CPSR e o contador de programa (PC).

1. Entradas:

- a) instruction: Instrução de 32 bits que define as operações a serem realizadas.
- b) peripheral_signal: Sinal que pode indicar eventos externos que afetam o controle do processador.
- c) clock: Sinal de clock que sincroniza as operações do módulo.

2. Saídas:

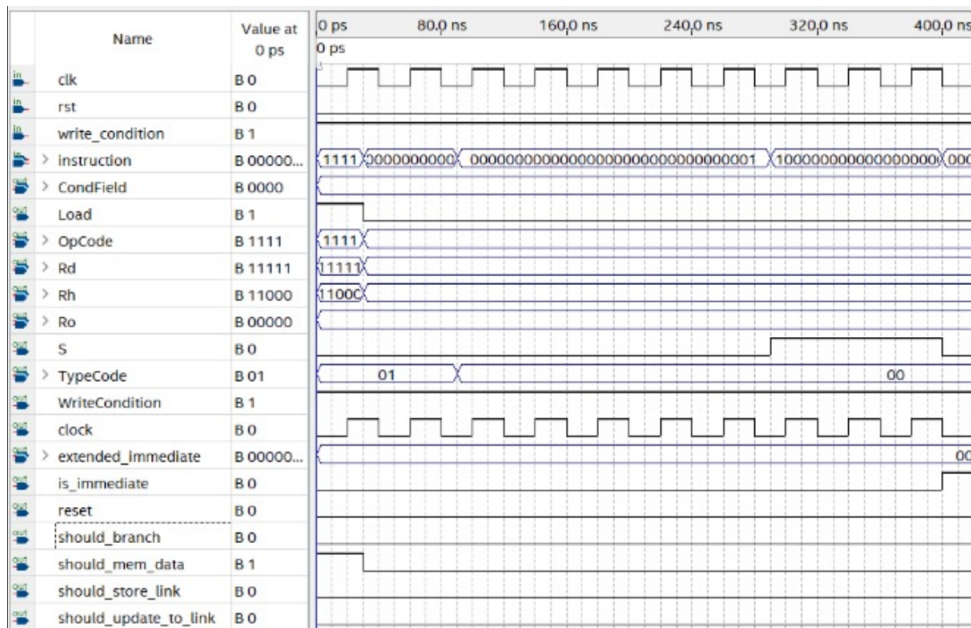
- a) halt_temporarily_signal: Sinal que indica se o processador deve pausar temporariamente suas operações.
- b) TypeCode: Código de 2 bits que define o tipo de operação a ser realizada.
- c) Load: Sinal que indica se a operação é de carga.
- d) should_store_link: Sinal que indica se o valor do link deve ser armazenado.
- e) Rh: Endereço do registrador fonte 1.
- f) Ro: Endereço do registrador fonte 2.
- g) Rd: Endereço do registrador de destino.
- h) extended_immediate: Valor imediato estendido para 32 bits.
- i) is_immediate: Sinal que indica se a operação utiliza um valor imediato.
- j) OpCode: Código de operação de 4 bits que define a operação a ser realizada pela ALU.
- k) should_use_data_memory: Sinal que indica se a memória de dados deve ser usada.
- l) CondField: Campo de condição de 4 bits para o módulo CPSR.
- m) set_cond_bit: Sinal que indica se o bit de condição deve ser ajustado.
- n) should_branch: Sinal que indica se o PC deve realizar uma ramificação.
- o) should_branch_to_link: Sinal que indica se o PC deve ramificar para o endereço de link.

3. Funcionalidade:

- a) A cada borda de subida do clock, o módulo verifica se o sinal `halt_temporarily_signal` e `peripheral_signal` estão ativos. Se sim, o sinal `halt_temporarily_signal` é desativado. Caso contrário, o módulo verifica se o `TypeCode` é 2'b00 e o `OpCode` é 4'b1001 para ativar o sinal `halt_temporarily_signal`.
- b) O módulo decodifica a instrução para gerar sinais de controle para o banco de registradores, a ALU, a memória de dados, o módulo CPSR e o PC. Isso inclui definir o `TypeCode`, `Load`, `should_store_link`, `Rd`, `Rh`, `Ro`, `OpCode`, `is_immediate`, `extended_immediate`, `should_use_data_memory`, `CondField`, `set_cond_bit`, `should_branch` e `should_branch_to_link`.

Para testar se o módulo funciona corretamente, foi utilizado a seguinte forma de onda:

Figura 20 – Forma de onda da unidade de controle parte 1



Fonte: O autor.


```
// Output CPSR Module
output [3:0] CondField,
output set_cond_bit, // Set Condition

// Output PC
output should_branch,
output should_branch_to_link
);
initial halt_temporarily_signal = 0;

always@(clock) begin
    if (halt_temporarily_signal & peripheral_signal)
        halt_temporarily_signal = 0;
    else
        if (TypeCode == 2'b00 & OpCode == 4'b1001)
            halt_temporarily_signal = 1;
end

// Register bank management:
assign TypeCode = instruction [27:26];
assign Load = instruction [20];
assign should_store_link = instruction[27] & instruction[26] & instruction[23];
assign Rd = instruction [19:15];
assign Rh = instruction [14:10];
assign Ro = instruction [9:5];

// ALU Control management
assign OpCode = instruction [23:20];
assign is_immediate = instruction [25];

assign extended_immediate = {{22{instruction[9]}}}, instruction[9:0]];

// Data Memory management
assign should_use_data_memory = ~instruction[27]
                                & instruction[26]
                                & instruction[20];
```

```
// CPSR Module management
assign CondField = instruction [31:28];
assign set_cond_bit = instruction [24];

// PC management
assign should_branch = instruction[27] & instruction[26];
assign should_branch_to_link = instruction[27]
                                & instruction[26]
                                & instruction[22];

endmodule
```

5.2.8 Controlador da ULA

O módulo ALUControl é responsável por configurar e controlar a operação da Unidade Aritmética e Lógica (ALU) com base nos códigos de operação e valores fornecidos. Ele determina o segundo operando da ALU e gera o resultado da operação, além de sinais de condição.

1. Entradas:

- a) OpCode: Código de operação de 4 bits que define a operação a ser realizada pela ALU.
- b) TypeCode: Código de 2 bits que especifica o tipo de operação a ser realizada.
- c) is_immediate: Sinal que indica se o valor do operando é imediato.
- d) immediate_value: Valor imediato de 32 bits que pode ser utilizado como operando.
- e) RhValue: Valor do registrador Rh, utilizado como primeiro operando da ALU.
- f) RoValue: Valor do registrador Ro, que pode ser utilizado como segundo operando da ALU.
- g) peripheral_value: Valor periférico de 32 bits que pode ser utilizado pela ALU.

2. Saídas:

- a) zero: Sinal que indica se o resultado da operação é zero.
- b) negative: Sinal que indica se o resultado da operação é negativo.
- c) result: Resultado da operação realizada pela ALU.
- d) r1_value: Valor do primeiro operando da ALU.
- e) r2_value: Valor do segundo operando da ALU.

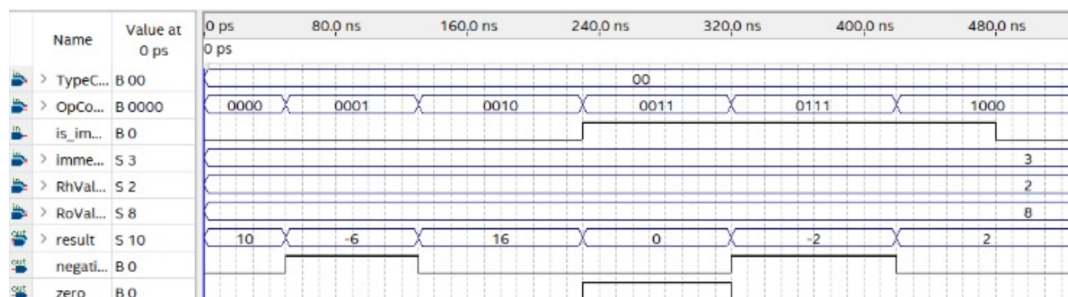
- f) operand2: Segundo operando utilizado pela ALU, que pode ser o valor imediato ou o valor do registrador Ro.

3. Funcionalidade:

- O módulo configura o valor do segundo operando (**operand2**) com base no sinal **is_immediate**. Se **is_immediate** estiver ativo, **operand2** é configurado para **immediate_value**. Caso contrário, **operand2** é configurado para **RoValue**.
- Em seguida, o módulo instancia uma ALU que realiza a operação com **RhValue** e **operand2**, além de considerar **peripheral_value**, **TypeCode** e **OpCode**.
- A ALU retorna o resultado da operação, bem como sinais de condição (**zero** e **negative**) e valores dos operandos utilizados (**r1_value** e **r2_value**).

Para testar se o módulo funciona corretamente, foi utilizado a seguinte forma de onda:

Figura 22 – Forma de onda da memória de instruções



Fonte: O autor.

O código final do controlador da ULA implementado foi:

```
module ALUControl (
    // Codes
    input [3:0] OpCode,
    input [1:0] TypeCode,
    input is_immediate,

    // Values
    input [31:0] immediate_value,
    input signed [31:0] RhValue,
    input signed [31:0] RoValue,
    input [31:0] peripheral_value,
```

```
// Outputs
output zero,
output negative,
output signed [31:0] result,
output signed[31:0] r1_value,
output signed [31:0] r2_value,
output reg [31:0] operand2
);
always@(is_immediate or immediate_value or RoValue) begin
    if (is_immediate)
        operand2 = immediate_value;
    else
        operand2 = RoValue;
end
alu alu (
    .A(RhValue),
    .B(operand2),
    .peripheral_value(peripheral_value),
    .TypeCode(TypeCode),
    .OpCode(OpCode),
    .result(result),
    .negative(negative),
    .zero(zero),
    .r1_value(r1_value),
    .r2_value(r2_value)
);
endmodule
```

5.2.8.1 ULA

O módulo `alu` é responsável por realizar operações aritméticas e lógicas sobre os operandos fornecidos, de acordo com o código de operação e o tipo de operação especificados. Ele também gera sinais de condição com base no resultado da operação.

1. Entradas:

- a) A: Operando A de 32 bits, utilizado na operação aritmética ou lógica.
- b) B: Operando B de 32 bits, utilizado na operação aritmética ou lógica.
- c) `peripheral_value`: Valor periférico de 32 bits, utilizado em operações específicas.
- d) `TypeCode`: Código de 2 bits que define o tipo de operação a ser realizada.

- e) OpCode: Código de operação de 4 bits que especifica a operação a ser realizada.

2. Saídas:

- a) result: Resultado da operação aritmética ou lógica.
- b) negative: Sinal que indica se o resultado da operação é negativo.
- c) zero: Sinal que indica se o resultado da operação é zero.
- d) r1_value: Valor do operando A (para testes).
- e) r2_value: Valor do operando B (para testes).

3. Funcionalidade:

- a) O módulo realiza a operação aritmética ou lógica com base no **TypeCode** e **OpCode**. Se **TypeCode** for 2'b00, a operação é definida pelo **OpCode**:
 - 4'b0000: Adição (**ADD**)
 - 4'b0001: Subtração (**SUB**)
 - 4'b0010: Multiplicação (**MUL**)
 - 4'b0011: Divisão (**DIV**)
 - 4'b0100: E lógico (**AND**)
 - 4'b0101: OU lógico (**OR**)
 - 4'b0110: XOR lógico (**XOR**)
 - 4'b0111: Negação (**NOT**)
 - 4'b1000: Movimento (**MOV**), onde o resultado é definido pelo operando B
 - 4'b1001: Valor periférico (**MOV peripheral_value**)
 - Caso contrário, o resultado é 32'b0.
- b) Se **TypeCode** não for 2'b00, o módulo simplesmente retorna o operando A como resultado (endereço para carga e armazenamento).
- c) O módulo avalia o resultado para determinar se ele é zero ou negativo, ajustando os sinais **zero** e **negative** conforme apropriado.
- d) O módulo também fornece os valores dos operandos A e B para fins de teste.

O código final da ULA implementado foi:

```
module alu (  
    input signed [31:0] A,  
    input signed [31:0] B,  
    input [31:0] peripheral_value,  
    input [1:0] TypeCode,
```

```

input [3:0] OpCode,
output reg signed [31:0] result,
output reg negative,
output reg zero,

// Test
output [31:0] r1_value,
output [31:0] r2_value
); // inputs and outputs
always @(A or B or TypeCode or OpCode) begin
    if (TypeCode == 2'b00) begin
        case (OpCode)
            4'b0000: result = A + B; // ADD
            4'b0001: result = A - B; // SUB
            4'b0010: result = A * B; // MUL
            4'b0011: result = A / B; // DIV
            4'b0100: result = A & B; // AND
            4'b0101: result = A | B; // OR
            4'b0110: result = A ^ B; // XOR
            4'b0111: result = -A; // NOT
            4'b1000: result = B; // MOV
            4'b1001: result = peripheral_value;
            default: result = 32'b0;
        endcase
    end else begin
        result = A; // Address for load and store
    end end

always@(result) begin
    if (result == 0)
        zero = 1;
    else
        zero = 0;
    if (result < 0)
        negative = 1;
    else
        negative = 0;
end
// Testing

```

```
assign r1_value = A;  
assign r2_value = B;  
endmodule
```

5.2.9 Divisor de frequência

O módulo `freqDiv` é um divisor de frequência que reduz a frequência do sinal de entrada `CLOCK_50` para um sinal de saída `clk` com uma frequência mais baixa.

1. Entradas:

- a) `CLOCK_50`: Sinal de clock de 50 MHz utilizado como entrada para o divisor de frequência.

2. Saídas:

- a) `clk`: Sinal de clock dividido com a frequência reduzida em relação ao sinal de entrada `CLOCK_50`.

3. Funcionalidade:

- a) O módulo utiliza um registrador `OUT` de 28 bits para contar os ciclos do sinal de entrada `CLOCK_50`.
- b) A cada borda de subida de `CLOCK_50`, o contador `OUT` é incrementado em 1.
- c) Quando `OUT` atinge o valor 50 (`27'd50`), o contador é reiniciado para 0 e o sinal `S` é definido como 1, indicando uma borda de subida no sinal de saída `clk`.
- d) Se `OUT` não tiver alcançado 50, o sinal `S` permanece 0.
- e) O sinal `clk` é atribuído ao valor de `S`, fornecendo um sinal de clock com uma frequência reduzida.

O código final do divisor de frequência implementado foi:

```
module freqDiv (CLOCK_50, clk);  
    input wire CLOCK_50;  
    reg S;  
    reg [27:0] OUT;  
    output clk;  
  
    always @ (posedge CLOCK_50)  
    begin  
        if(OUT == 27'd50)
```

```
begin
    OUT = 0;
    S = 1;
end
else begin
    OUT = OUT + 1;
    S = 0;
end
end
assign clk = S;
endmodule
```

5.2.10 Módulo integrador

O módulo `integrated` é um sistema integrado que coordena diversas unidades funcionais para realizar operações de um processador. Ele conecta e gerencia as interações entre diferentes módulos, incluindo controle de unidade, memória, registradores e unidades aritméticas e lógicas.

1. Entradas:

- a) `clock`: Sinal de clock do sistema.
- b) `reset`: Sinal para reiniciar o sistema.
- c) `peripheral_signal`: Sinal de entrada de periférico.
- d) `peripheral_value`: Valor do periférico a ser usado nas operações.

2. Saídas:

- a) `write_condition`: Sinal que indica quando uma escrita deve ocorrer.
- b) `output0`, `output1`, `output2`, `output3`: Sinais de saída com valores internos do sistema.

3. Funcionalidade:

- a) O módulo conecta diferentes blocos funcionais, como `ControlUnit`, `PC_main`, `InstructionMemory`, `registerBank`, `ALUControl`, `CPSR_module`, e `DataMemory`.
- b) Cada bloco é responsável por uma parte específica do processamento, como controle de fluxo, execução de instruções, manipulação de registradores e memória.
- c) O módulo coordena a comunicação entre esses blocos para garantir o funcionamento correto do processador integrado.

O código final do módulo integrador implementado foi:

```
module integrated (
    input clock,
    input reset,
    input peripheral_signal,
    input [31:0] peripheral_value,

    output write_condition,
    output [31:0] output0,
    output [31:0] output1,
    output [31:0] output2,
    output [31:0] output3
);
    assign output0 = r31_value;
    assign output1 = r0_value; // Won't be used
    assign output2 = r0_value;
    assign output3 = current_instr_addr;

    // Values
    wire [31:0] extended_immediate;
    wire [31:0] current_link_value;
    wire [31:0] alu_result;
    reg [31:0] write_in_reg;
    wire [31:0] rh_value;
    wire [31:0] ro_value;
    wire [31:0] operand2;

    wire [31:0] op2_value;
    wire [31:0] r0_value;
    wire [31:0] r31_value;

    // Codes
    wire [1:0] TypeCode;
    wire Load_bit;
    wire should_store_link;
    wire is_immediate;
    wire [3:0] OpCode;
    wire should_use_data_memory;
```

```
wire [3:0] CondField;
wire set_cond_bit;
wire should_branch;
wire should_branch_to_link;
wire [31:0] current_instr_code;
wire zero_bit;
wire neg_bit;
wire should_write_using_ram;
wire halt_temporarily_signal;

// Addresses
wire [4:0] rh_addr;
wire [4:0] ro_addr;
wire [4:0] rd_addr;
wire [31:0] current_instr_addr;

// UC
ControlUnit integrated0(
    .instruction(current_instr_code),
    .peripheral_signal(peripheral_signal),
    .halt_temporarily_signal(halt_temporarily_signal),
    .clock(clock),

    // Output register bank
    .TypeCode(TypeCode),
    .Load(Load_bit),
    .should_store_link(should_store_link),
    .Rh(rh_addr),
    .Ro(ro_addr),
    .Rd(rd_addr),

    // Output ALU Control
    .extended_immediate(extended_immediate),
    .is_immediate(is_immediate),
    .OpCode(OpCode),

    // Output Data Memory
    .should_use_data_memory(should_use_data_memory),
```

```

// Output CPSR Module
.CondField(CondField),
.set_cond_bit(set_cond_bit), // Set Condition

// Output PC
.should_branch(should_branch),
.should_branch_to_link(should_branch_to_link)
);

```

```

PC_main integrated1(
.branch_value(alu_result), // ALU Result
.link_value(current_link_value), // RegBank

.should_branch(should_branch),
.write_condition(write_condition),
.should_branch_to_link(should_branch_to_link),

.halt_temporarily_signal(halt_temporarily_signal),
.clock(clock),
.reset(reset),
.instruction_address(current_instr_addr)
);

```

```

InstructionMemory integrated2(
.addr(current_instr_addr),
.clock(clock),
.instruction(current_instr_code),
.first_instr_line_test() // Test
);

```

```

registerBank integrated3( // Inputs are Rh, Ro (Op2), Clock, TypeCode, WriteData (
// Addresses
.RhAddress(rh_addr),
.RoAddress(ro_addr),
.RdAddress(rd_addr),

// Values
.WriteData(write_in_reg),
.new_LinkValue(current_instr_addr),

```

```
// Codes
.Load(Load_bit),
.TypeCode(TypeCode),
.write_condition(write_condition),
.should_store_link(should_store_link),

// Clock
.clock(clock),
// Outputs
.RhValue(rh_value),
.RoValue(ro_value),
.RdValue(), // Testing
.LinkValue(current_link_value),

// Test
.r0_value(r0_value),
.r31_value(r31_value)
);

ALUControl integrated4(
// Codes
.OpCode(OpCode),
.TypeCode(TypeCode),
.is_immediate(is_immediate),

// Values
.immediate_value(extended_immediate),
.RhValue(rh_value),
.RoValue(ro_value),
.peripheral_value(peripheral_value),

// Outputs
.zero(zero_bit),
.negative(neg_bit),
.result(alu_result),
.operand2(operand2),

.r1_value(), // A
```

```

    .r2_value(op2_value) // B
);

CPSR_module integrated5(
    .ALUresult(alu_result),
    .set_cond_bit(set_cond_bit),
    .zero(zero_bit),
    .negative(neg_bit),
    .Cond_field(CondField),
    .clock(clock),
    .write_condition(write_condition),
    .current_flags() // Test
);

DataMemory integrated6(
    .new_data(operand2),
    .mem_addr(alu_result),
    .Load(Load_bit),
    .clock(clock),
    .write_condition(write_condition),
    .TypeCode(TypeCode),
    .ram_output(ram_output),
    .data_saved() // Test
);

assign should_write_using_ram = TypeCode[0] & ~TypeCode[1] & ~Load_bit;

always@(should_write_using_ram or ram_output or alu_result) begin
    if (should_write_using_ram)
        write_in_reg = ram_output;
    else
        write_in_reg = alu_result;
    end
endmodule

```

5.2.11 Módulo final do processador

O módulo **Processor** é responsável por integrar e exibir os resultados do processador em uma placa de desenvolvimento. Ele utiliza um clock dividido e controla a exibição dos resultados em múltiplos displays de 7 segmentos.

1. Entradas:

- a) `CLOCK_50`: Clock de 50 MHz.
- b) `KEY`: Chaves para controle.
- c) `SW`: Interruptores para configuração.

2. Saídas:

- a) `LEDR`: LED indicador.
- b) `HEX0` a `HEX7`: Displays de 7 segmentos para visualização dos resultados.

3. Funcionalidade:

- a) O módulo `freqDiv` divide o clock de 50 MHz para gerar um clock mais lento de 1 MHz.
- b) O sinal `true_clock` é usado como entrada de clock para o módulo `integrated`, que gerencia o processamento e a lógica interna.
- c) O módulo `integrated` fornece quatro saídas (`output0` a `output3`), que são conectadas a diferentes módulos `output_module`.
- d) Os módulos `output_module` convertem os valores das saídas em sinais para os displays de 7 segmentos (`HEX0` a `HEX7`), com cada módulo configurado para exibir uma parte específica das saídas.

O código final da memória de instruções implementado foi:

```
module Processor(  
    input CLOCK_50,  
    input [3:0] KEY,  
    input [17:0] SW,  
  
    output [0:0] LEDR,  
    output [6:0] HEX0,  
    output [6:0] HEX1,  
    output [6:0] HEX2,  
    output [6:0] HEX3,  
    output [6:0] HEX4,  
    output [6:0] HEX5,  
    output [6:0] HEX6,  
    output [6:0] HEX7  
);
```

```
freqDiv freqDiv (
    .CLOCK_50(CLOCK_50),
    .clk(divided_clock)
); // Clock divided by 50 into 1MHz frequency

wire true_clock;
assign true_clock = divided_clock & ~SW[17];
wire [31:0] output0;
wire [31:0] output1;
wire [31:0] output2;
wire [31:0] output3;

integrated modules( // All modules integrated
    .clock(true_clock),
    .reset(SW[16]),
    .peripheral_signal(SW[15]),
    .peripheral_value(SW[4:0]),
    .write_condition(LED_R[0]),
    .output0(output0),
    .output1(output1),
    .output2(output2),
    .output3(output3),
);

output_module output_group0( // Answer
    .number(output0),

    .HEX0(HEX0),
    .HEX1(HEX1),
    .HEX2(HEX2),
    .HEX3(HEX3)

);

output_module output_group1( // null
    .number(output1),

    .HEX0(),
    .HEX1(),
    .HEX2(),
```

```
.HEX3()
);
output_module output_group2( // Iteration Counter / Current State
    .number(output2),

    .HEX0(HEX4),
    .HEX1(HEX5),
    .HEX2(),
    .HEX3()
);
output_module output_group3( // Current Instr index
    .number(output3),

    .HEX0(HEX6),
    .HEX1(HEX7),
    .HEX2(),
    .HEX3()
);
endmodule
```

5.3 Assembler

Para facilitar os testes finais do desenvolvimento do processador, foi desenvolvido um código em *Python* que leia um código de linguagem interpretável em linguagem de máquina (binário) (PYTHON, 2024). Isso facilitou os testes, pois os códigos de máquina puderam ser facilmente substituídos a depender das necessidades de teste e a falta de erros na contagem de bits permitiu um *debugging* mais eficiente (extração e identificação de erros).

O código completo e histórico de edições pode ser verificado em <https://github.com/HeitorReis/ARM_assembler/>.

Figura 23 – Código em assembler exemplo - a ser transformado em linguagem de máquina

```

1  movi: r1 = 0
2  movi: r2 = 1
3  movi: r3 = 8
4  mov: r0 = r3
5  add: r2 = r1, r2
6  sub: r1 = r2, r1
7  subi: r3 = r3, 1
8  subis: r16 = r3, 0
9  bineq: -7
10 mov: r0 = r3
11 mov: r31 = r2
12 bi: -1

```

Fonte: O autor.

Figura 24 – Código em linguagem de máquina exemplo - transformado a partir do código assembler

```

1  00000010100000001000000000000000
2  00000010100000001000000000000001
3  000000101000000011000000000001000
4  00000000100000000000000001100000
5  000000000000000010000010001000000
6  000000000000100001000100000100000
7  00000010000100011000110000000001
8  000000110001100000000110000000000
9  001011100000000000000001111111001
10 000000001000000000000000011000000
11 000000001000111110000000010000000
12 000011100000000000000001111111111

```

Fonte: O autor.

Descobrir como descrever a linguagem de máquina de um processador novo é uma tarefa difícil, pois ela reflete, às vezes, em como o próprio processador vai funcionar. Desta forma, algumas barreiras foram encontradas pelo autor durante a programação do assembler, mas foi alcançado um resultado significativo que funciona em duas etapas principais: Desmembramento (*disassembler*), que separa os *tokens* do código em assembler;

e Decodificador (*decoder*), que decodifica cada token em seu respectivo campo dentro da instrução em código de máquina.

5.4 Testes em bancada

Por fim, foram realizados testes em bancadas juntando os produtos das três últimas seções e finalmente implementando eles em uma placa FPGA. Foi consultado o manual da placa para obter-se os códigos de pinagem de cada chave e led dos displays da placa (TERASIC, 2012).

O vídeo do primeiro teste em bancada pode ser acessado em <https://youtu.be/fZlr4-7wM_8>.

Estes testes demonstraram algumas dificuldades. A pinagem do FPGA nem sempre era certa, por conta de erros humanos, o que dificultava encontrar o erro da implementação. Algumas vezes, o resultado incorreto não era suficiente para encontrar o erro no código em verilog, o que demandava várias compilações com pequenas alterações até que o erro fosse encontrado e o código corrigido.

6 Resultados

Um dos principais resultados do projeto foi a implementação bem-sucedida do processador na placa FPGA, o que confirmou a funcionalidade do design. A eficácia dessa implementação pode ser verificada no vídeo da seção [seção 5.4](#).

Além disso, foi desenvolvido um assembler em *Python* (PYTHON, 2024), que desempenha um papel crucial na interação com o processador. Este assembler opera em duas etapas principais: desmembramento (*disassembler*), que organiza os *tokens* do código em assembler, e decodificação (*decoder*), que traduz esses *tokens* em instruções de máquina. Esta ferramenta é fundamental para a programação e teste do processador, demonstrando a aplicação prática e o sucesso do projeto.

7 Conclusão

O desenvolvimento deste projeto apresentou uma série de desafios que testaram a compreensão e as habilidades na área de Engenharia de Computadores. A criação de um assembler para o processador revelou complexidades consideráveis. O processo de desmembramento (*disassembler*) e decodificação (*decoder*) das instruções exigiu ajustes contínuos para garantir a precisão na conversão de código de assembler para código de máquina.

Além disso, a implementação na placa FPGA trouxe seus próprios obstáculos. A pinagem do FPGA frequentemente apresentava problemas devido a erros humanos, o que dificultava a identificação e correção de falhas. A complexidade do desenvolvimento em Verilog exigiu múltiplas compilações e ajustes minuciosos para detectar e corrigir erros, refletindo o grau de atenção necessário para a criação de sistemas de hardware.

Apesar desses desafios, a implementação bem-sucedida do processador na placa FPGA e o desenvolvimento do assembler em Python demonstraram a robustez e a funcionalidade do design proposto. A integração eficaz dos módulos, incluindo a unidade de controle, o banco de registradores, a memória de instruções, o ALU e o módulo de controle da CPSR, comprova a precisão e a eficácia do processador desenvolvido.

Dado o crescente investimento e destaque nas pesquisas e inovações em Engenharia de Computadores e Projeto de Processadores, é fundamental o estudo de modelos atuais de arquitetura de processadores. A prática desses conceitos não apenas enriquece, mas também capacita futuros profissionais para atuação no mercado de trabalho e em ambientes laboratoriais. Este projeto reflete o domínio dos conceitos de arquitetura de processadores e destaca a relevância das técnicas modernas no design e construção de sistemas computacionais. A capacidade de implementar e testar um processador funcional sublinha a importância da educação prática e da inovação na formação de engenheiros de computação.

Referências

ARM. *ARM7TDMI-S Technical Reference Manual*. [S.l.]: ARM, 2001. Citado 2 vezes nas páginas 12 e 13.

ARM. *Arm A-profile Architecture Registers*. 2024. Último acesso em: Abril de 2024. Disponível em: <<https://developer.arm.com/documentation/ddi0601/2023-12/AArch32-Registers/CPSR--Current-Program-Status-Register>>. Citado 2 vezes nas páginas 12 e 20.

ARM7TDMI-S Data Sheet. 3. ed. [S.l.]: ARM, 1995. Citado 3 vezes nas páginas 12, 13 e 14.

HARRIS, S. L. *Digital Design and Computer Architecture*. 1. ed. UK: Elsevier, 2016. Citado na página 10.

HENNESSY JOHN L.

PATTERSON, D. A. *Computer Organization and Design: The Hardware/Software Interface*. [S.l.]: Morgan Kaufman, 2014. Citado 3 vezes nas páginas 9, 22 e 23.

INTEL. *Quartus Prime Lite Edition*. 2019. Citado 2 vezes nas páginas 2 e 28.

MOORE, S. *MIPS DataPath Lecture*. 2015. Último acesso em: Março de 2024. Disponível em: <<https://www.youtube.com/watch?v=YGSAWqQy9bI>>. Citado na página 11.

PYTHON. *Python*. 2024. Último acesso em: Setembro de 2024. Disponível em: <<https://www.python.org>>. Citado 2 vezes nas páginas 65 e 68.

REIS, H. *GitHub do projeto*. 2024. Último acesso em: Setembro de 2024. Disponível em: <https://github.com/HeitorReis/ARM_processor/settings>. Citado na página 28.

REIS, H. *Site do autor*. 2024. Último acesso em: Setembro de 2024. Disponível em: <<https://hbreis.pythonanywhere.com>>. Citado na página 24.

TERASIC TECHNOLOGIES INC. *Terasic DE2-115 User Manual*: World leading fpga based products and design services. [S.l.], 2012. 122 p. Citado 3 vezes nas páginas 2, 8 e 67.

TMLT, E. da. *10 diferenças entre Von Neumann e Harvard architecture*. 2021. Acesso em: 14 maio 2022. Disponível em: <<https://www.currentschoolnews.com/pt/not%C3%ACcias-de-educa%C3%A7%C3%A3o/diferen%C3%A7a-entre-a-arquitetura-de-von-neumann-e-harvard/>>. Citado na página 9.